AST Midterm Study Notes

Oct 12, 2026 · @Arham

SESD3263 Automated Software Testing, UCP Lahore, Fall 2026. Built only from your outline, Week 1–3 lecture files and handouts, and standard core concepts, with no outside references.

Read this first

These notes follow your outline's Weeks 1 to 7 (the midterm covers up to Week 7). Four things you should know before you rely on them:

1. **Your code was not attached.** I received the description of `week_3_ATS` and `Locators`, but not the Java files. Part 2 therefore uses a **reconstructed version** that matches your description and the bugs you listed. Paste your real files and I will match it line by line.
2. **Only Weeks 1 to 3 have lecture files.** Week 4 (element interaction), Week 5 (Actions, waits), Week 6 (TestNG assertions, XML, reports) and Week 7 (POM) are in the outline only. I wrote those from standard core concepts and marked them *(outline only)*. Check them against your teacher's slides when they arrive.
3. **The midterm is practical.** The outline says "Midterm Practical Examination" in Week 8. So the code examples and practice tasks matter more than the MCQs in Part 3.
4. **Two slide statements are wrong or unclear.** Week 3 slide 10 says `findElement()` returns `null` when nothing matches. In real Selenium it **throws** **`NoSuchElementException`**; only `findElements()` returns an empty list. Also, Week 2.2 handout uses `#` for comments in Java code; Java uses `//`. Write the correct form in the exam.

Course item	Value from your files
Course	SESD3263 Automated Software Testing, 3 credit hours
Instructor	Hira Asim
Tools	Katalon Recorder, IntelliJ, Postman, JMeter
Textbook	Garcia, *Hands-On Selenium WebDriver with Java*, O'Reilly 2022
Marks	Class activity 5%, quizzes 20%, assignments 10%, midterm 25%, final 40%

Course item	Value from your files
Your dependencies	selenium-java 4.50.0 (compile), testng 7.12.0 (test)
Handout dependencies	selenium-java 4.39.0, testng 7.8.0 (versions differ; both work the same way)

PART 1: TOPIC-WISE NOTES

Every topic has the same four parts: concept, why it is needed, key syntax, and common mistakes.

1.1 Introduction to Automation Testing (Week 1)

Concept. *Software testing* is the process of checking that a program meets its requirements by running it to find defects. *Automation testing* means using tools and scripts to run test cases by themselves, compare the **actual result** with the **expected result**, and report pass or fail. Tools named in your slides: Selenium, Cypress, Katalon Studio.

Why it is needed. Some tests are repeated after every software change. This is called *regression testing* (checking that old features still work after a change). A script can repeat 200 such tests in minutes and never gets bored or skips a step. Slide example: running 200 Daraz regression tests in minutes instead of hours.

Workflow. The Week 1 slide titled "Automation testing workflow" is a picture and I could not read it. The standard workflow is: pick test cases worth automating, choose a tool, set up the environment, write the scripts, run them, analyse the results, then maintain the scripts. Confirm the exact steps against your slide.

Key syntax. No code is needed at this level; the idea is just *expected vs actual*:

```
String expected = "Home Page";            // what the requirement says
String actual   = driver.getTitle();      // what the app really shows
boolean passed  = expected.equals(actual); // automation compares them for
you
```

Benefits (slide 8)	Challenges (slide 9)
Faster execution	Initial setup time and cost
Increased coverage (many browsers, screen sizes)	Maintenance overhead (UI change means script update)

Benefits (slide 8)	Challenges (slide 9)
Reusability across releases	Not suitable for UI/UX feel ("is it user-friendly?")
Reliability, same steps every time	Skilled workforce needed (Java or Python)
Cost-effective over many runs	False positives and negatives (a slow network fails a test, no real bug)

Common mistakes.

- Believing automation replaces manual testing. It does not; it covers repetitive checks only.
- Saying automation is always cheaper. It has a **higher starting cost**.
- Treating every failed test as a real bug. A *false positive* is a test failure that is not a defect.

1.2 Manual vs Automated Testing (Week 1)

Concept. In *manual testing* a person follows test cases by hand and uses the app like an end user (example: composing and sending a Gmail message). In *automation testing* scripts do the same steps.

Why it matters. Choosing the right one saves time. Use automation for stable, repetitive, regression work. Use manual testing for new features and for judging look and feel.

Aspect	Manual testing	Automation testing
Execution	Tester performs steps	Scripts perform steps
Time	Slower for repeated tests	Faster for repeated tests
Cost	Lower initial cost	Higher initial cost (scripts, tools, skills)
Flexibility	Easy to change and explore	Script needs updating after a redesign
Human error	More likely when repetitive	Fewer repetitive mistakes
Reusability	Repeated by hand each release	Scripts reused every release

Key syntax. Not applicable. Remember the table above; exams often ask for 3 to 5 differences.

Common mistakes. Writing "automation has no cost", or forgetting that scripts need **maintenance** whenever the page changes.

1.3 Katalon Recorder (Week 1)

Concept. Katalon Recorder is a light browser extension (Chrome, Firefox, Edge) for **record-and-playback** testing. You do the steps once; it records them as commands and can replay them. It can also **export** a test to Selenium WebDriver or Katalon Studio.

Why it is needed. It lets beginners automate a simple test with almost no programming, and it is a quick way to see which locators a page offers.

Key features (slide 13). Record and playback, assertions (verify an element or page state), export, and test suites (several test cases run together).

Key syntax. A recorded test is a table of commands. A typical login test looks like this:

Command	Target	Value
open	https://automationexercise.com	
click	link=Signup / Login	
type	name=email	sample@user.com
type	name=password	123456789
click	xpath=//button[@type='submit']	

The exact command names and targets above are a typical layout, not copied from your slides. Check them in the tool.

Limitations (slides 19 and 20) and what Selenium offers instead.

Katalon Recorder limit	Selenium benefit
No loops or conditions	Full Java or Python logic
Weak on file upload, multiple windows, pop-ups	Handles windows, uploads, downloads
No parallel execution	Selenium Grid runs tests on many browsers together
Basic reports, no failure screenshots	Reporting frameworks such as Extent Reports

Debugging. Common errors are unrecognised elements or commands and wrong XPath or CSS locators. Use pause, resume and step-through, and edit the recorded step.

Common mistakes. Trusting recorded locators blindly (they can be fragile), and forgetting to save the test case or suite before closing.

1.4 Selenium and WebDriver Architecture (Week 2)

Concept. *Selenium* is a free automation tool used mainly to test web applications. It works on many operating systems and browsers, and you can write tests in Java, Python, Kotlin, JavaScript and more. *Selenium WebDriver* is the part of Selenium that gives you a programming interface to control a browser and its elements (buttons, text boxes, links).

How the pieces talk. Your script does not touch the browser directly. A browser-specific *driver* sits in between (slide 10: two-way communication through, for example, ChromeDriver):

```
Your test script (Java + TestNG)
        |  calls Selenium client library (WebDriver API)
        v
Browser driver  (ChromeDriver, GeckoDriver for Firefox)
        |  sends commands, receives results
        v
Browser (Chrome, Firefox)  ->  DOM of the web page
```

The commands travel between library and driver over the standard WebDriver protocol (HTTP messages). Your slides do not go into that detail, so treat the protocol name as background knowledge.

Why it is needed. One script can run on different browsers (cross-browser support), in different languages, and can be repeated after every release. That is why WebDriver supports continuous integration and delivery.

Key syntax. Cross-browser choice (slide 7):

```
WebDriver driver;                    // declare using the interface
String browser = "firefox";
if (browser.equals("chrome")) {
    driver = new ChromeDriver();     // controls Chrome
} else if (browser.equals("firefox")) {
    driver = new FirefoxDriver();    // controls Firefox
}
```

Common mistakes.

- Mixing up *Selenium* (the whole suite) with *WebDriver* (one component).
- Thinking Selenium "sees" the page like a human. It works on the **DOM**, not on pixels.
- Forgetting that you need a matching driver. Your slides mention ChromeDriver and GeckoDriver. Recent Selenium versions (from 4.6) can fetch the driver for you through *Selenium Manager*, which is why `new ChromeDriver()` works in your handout with no extra setup.

1.5 Maven, TestNG and WebDriver Setup (Week 2)

Concept. *Maven* is a build automation and project management tool for Java. You list the libraries you need in one file, **`pom.xml`**, and Maven downloads and manages them. *TestNG* is a testing framework that helps you organise and run test cases.

Why it is needed. Without Maven you would download jar files by hand and fix version problems yourself. With Maven, adding a `<dependency>` block is enough.

Project steps (Week 2.1 handout).

1. Install JDK (IntelliJ can offer it) and IntelliJ IDEA.
2. File > New > Project > Maven. Optionally set Group ID (for example `com.example`) and Artifact ID.
3. Check the folders: `src/main/java` (application code, rarely used in a test project), `src/test/java` (your test code), and `pom.xml`.
4. Paste the Selenium and TestNG dependencies inside `<dependencies>` after `</properties>`, then click the Maven reload icon.
5. Create package `tests` and class `FirstTest` under `src/test/java`, then run it.

Key syntax: the dependencies from your file.

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.50.0</version>
    <scope>compile</scope>   <!-- available everywhere -->
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
    <scope>test</scope>      <!-- only in src/test/java -->
  </dependency>
</dependencies>
```

Scope says where the library is available. `test` scope means TestNG is visible only to code under `src/test/java`.

Key syntax: the first test (Week 2.1).

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.testng.annotations.*;

public class FirstTest {
    WebDriver driver;                                  // reference only, no
browser yet

    @BeforeClass
    public void setup() {
        driver = new ChromeDriver();                   // opens Chrome
        driver.manage().window().maximize();           // full-size window
    }

    @Test
    public void testcase() {
        driver.get("https://mvnrepository.com/"); // open the page
        System.out.println("Title: " + driver.getTitle());
    }

    @AfterClass
    public void teardown() {
        driver.quit();                                 // close all windows, end
session
    }
}
```

Common mistakes.

- Forgetting to reload Maven after editing `pom.xml`, so the imports stay red.
- Putting test classes in `src/main/java`. TestNG has `test` scope, so it will not compile there.
- Declaring `WebDriver driver;` but never writing `driver = new ChromeDriver();`. The result is a `NullPointerException`.
- Using `close()` at the end and leaving the session open. Use `quit()` (see section 1.7).
- Slide and handout typo: the handout says Run 'FirstClass' but the class is `FirstTest`. Run the class you created.

1.6 Browser Navigation (Week 2.2)

Concept. Navigation commands open pages and move through the browser history, like the address bar and the back, forward and refresh buttons.

Why it is needed. Real tests visit several pages. For example: open the home page, go to a product, check it, go back, and make sure the home page is still correct.

Key syntax.

Command	What it does
`driver.get("url")`	Opens a URL in the current window
`driver.navigate().to("url")`	Also opens a URL (same result as `get` for a normal page)
`driver.navigate().back()`	One page back in history
`driver.navigate().forward()`	One page forward in history
`driver.navigate().refresh()`	Reloads the current page
`driver.getTitle()`	Returns the page title as a `String`
`driver.getCurrentUrl()`	Returns the current URL as a `String`

```
driver.get("https://www.pakwheels.com/");       // page 1
driver.navigate().to("https://www.daraz.pk");   // page 2
driver.navigate().back();                       // back on pakwheels
driver.navigate().forward();                    // forward on daraz
driver.navigate().refresh();                    // reload daraz
System.out.println(driver.getCurrentUrl());
```

Common mistakes.

- Calling `navigate().to("")` with an **empty string**. There is no page to open. In many setups this throws an exception; at best it does nothing useful. Always pass a full URL starting with `https://`.
- Calling `back()` when there is no earlier page. The browser simply stays where it is.
- Forgetting that `get()` and `navigate().to()` do **not** add a second browser tab. They reuse the same one.
- Writing the URL without `https://`.

1.7 Window and Tab Handling (Week 2.2 and Week 5 outline)

Concept. A browser can have several windows or tabs. Selenium gives each one a unique text ID called a **window handle**. Selenium controls only **one** window at a time (the *current* one). To control another, you must **switch** to its handle.

Why it is needed. Links often open in a new tab, and some tests need a second tab on purpose. If you do not switch, your commands still go to the old tab and the test fails or checks the wrong page.

Key syntax.

Command	What it does
`driver.getWindowHandle()`	Handle (a `String`) of the **current** window or tab
`driver.getWindowHandles()`	All open handles, returned as a `Set<String>`
`driver.switchTo().newWindow(WindowType.TAB)`	Opens a new blank tab **and moves focus to it** (Selenium 4)
`driver.switchTo().newWindow(WindowType.WINDOW)`	Same, but a new window
`driver.switchTo().window(handle)`	Moves focus to the window with that handle
`driver.close()`	Closes the **current** window or tab only
`driver.quit()`	Closes **all** windows and ends the session

```
String original = driver.getWindowHandle();          // 1. save the first
tab's handle

driver.switchTo().newWindow(WindowType.TAB);          // 2. new tab, focus
moves to it
driver.get("https://www.daraz.pk");                   // 3. loads in the NEW
tab

driver.close();                                       // 4. closes the new
tab only
driver.switchTo().window(original);                   // 5. MUST switch back;
focus is lost
System.out.println(driver.getTitle());                // works on the first
tab again
```

close() vs quit().

	`close()`	`quit()`
Closes	Current window or tab	Every window opened by the session
Session	Still alive if other windows remain	Ended; the driver process stops
Use it	Inside a test, to drop one tab	In `@AfterClass` or `@AfterTest` clean-up

Common mistakes.

- Not saving the original handle before opening a second tab, so you cannot return to it.
- Calling `driver.close()` and then using `driver` without `switchTo().window(...)`. Selenium throws `NoSuchWindowException` because the current window is gone.
- Using `close()` as the final clean-up. If more than one window is open, the browser and the driver process stay behind.
- Believing a handle is the same on every run. It changes each time, so always read it with `getWindowHandle()`; never type it by hand.
- Assuming `newWindow()` leaves you on the old tab. It already switches you to the new one.

1.8 DOM and Web Element Locators (Week 3)

Concept: the DOM. *DOM* means Document Object Model. The browser reads the page's HTML and builds a tree of objects from it. Selenium works on this tree, not on what a human sees. Your slide's picture: **HTML code, then DOM, then web page**. Building analogy: building = page, rooms = elements, room numbers = attributes (`id` , `name` , `class`), connections between rooms = parent-child relations.

Concept: elements and attributes. An *element* is made of an opening tag, content and a closing tag. In `<button id="loginBtn">Login</button>` , the tag is `button` , the attribute is `id="loginBtn"` , and the text is `Login` .

Concept: locator. A *locator* tells Selenium **which exact element** you mean. The DOM tells us what exists; a locator tells Selenium how to find it. A locator can be an ID, name, class name, tag name, link text, partial link text, CSS selector or XPath expression.

Why it is needed. A human thinks "click Login". Selenium needs `driver.findElement(By.id("loginBtn")).click();` . A poor locator is the most common reason an automated test fails.

Inspecting. Open the page, right-click the element, choose **Inspect**, and read its tag, attributes and text. Ask: *what information could I use to identify this element?*

Key syntax: all locator types (Week 3 handout).

Locator	Java syntax	Notes from the handout
ID	`By.id("sale_image")`	Fastest and most reliable; IDs should be unique
Name	`By.name("email")`	Good for form fields; use when ID is missing
Class name	`By.className("features_items")`	Fails for compound classes (names with spaces); use CSS or XPath then
Tag name	`By.tagName("input")`	Usually returns many elements; use with `findElements`
Link text	`By.linkText("Contact us")`	Only for `<a>` tags; exact visible text
Partial link text	`By.partialLinkText("art")`	Part of the text; may match the wrong link if not unique
CSS selector	`By.cssSelector("input[name='email']")`	Flexible, attribute based
XPath	`By.xpath("//input[@id='email']")`	Powerful, but slower and more brittle

Which locator should I use? (handout table)

Situation	Preferred locator
Unique id available	ID
Unique name available	Name
Unique class available	Class name
Identify a hyperlink by visible text	Link text
Flexible attribute-based locator	CSS
No useful unique attribute	XPath
Partial or dynamic matching	CSS or XPath
Relationship-based identification	XPath

CSS selector cheat sheet.

Goal	Syntax	Example
By tag	`tag`	`input`
By id	`#id`	`#email`
By class	`.class`	`.form-control`
By attribute equals	`tag[attr='v']`	`input[name='userEmail']`
Contains	`[attr*='v']`	`input[placeholder*='Email']`
Starts with	`[attr^='v']`	`input[placeholder^='Sub']`
Ends with	`[attr$='v']`	`textarea[placeholder$='Message']`
Multiple attributes	`tag[a='1'][b='2']`	`input[type='text'][id='email']`
Attribute exists	`tag[attr]`	`input[name]`

XPath cheat sheet. XPath means *XML Path Language*. Basic form: `//tagname[@attribute='value']`.

Goal	XPath
All elements of a tag	`//input`
By attribute	`//input[@id='email']`
Any tag, by attribute	`//*[@name='search']`
Two attributes	`//input[@type='text' and @name='email']`
Exact text	`//button[text()='Login']`
Contains	`//input[contains(@placeholder,'Name')]`
Starts with	`//h2[starts-with(text(),'Get')]`
Child of a parent	`//*[@class='productinfo text-center']/img`

findElement vs findElements.

```
WebElement one = driver.findElement(By.id("email"));        // first match,
or exception
List<WebElement> all = driver.findElements(By.tagName("input")); // every
match, or empty list
System.out.println("Inputs: " + all.size());
```

If `findElement` finds nothing it throws `NoSuchElementException`. (Your Week 3 slide 10 says it returns `null`; that is not how Selenium behaves. `findElements` returns an **empty list**.)

Common mistakes.

- Using `By.className` with a value that contains spaces such as `"productinfo text-center"`. Use CSS (`.productinfo.text-center`) or XPath.
- Choosing a locator that matches many elements, then acting on the first match without noticing.
- Using `linkText` on something that is not an `<a>` tag, or ignoring upper and lower case in the visible text.
- Writing long absolute XPaths that break when the page layout changes. Prefer attribute-based ones.
- Quote errors in XPath inside Java: use `"//input[@id='email']"` (single quotes inside double quotes).
- Copying a locator without testing it in Developer Tools first (Ctrl+F in the Elements tab accepts XPath and CSS).
- Typing the handout's locator values from memory. The handout itself shows `test_case_list` in one line and `test_cases_list` in a table, so always copy the value from the live page.
- The handout's "Add to cart" button example uses `//a[text()='Add to cart']`, which is an `<a>` tag, not a `<button>`. Check the real tag before writing the XPath.

1.9 Web Element Interaction (Week 2.2 handout; Week 4 outline only)

Concept. After you locate an element, Selenium returns a `WebElement` object. You call methods on it to act like a user: type, click, read text, or check its state. Week 4 of the outline adds text fields, buttons, links, checkboxes, radio buttons, dropdowns and lists.

Why it is needed. Locating alone proves nothing. A test must do something (type, click, upload) and then check the result.

Key syntax: methods in your handout.

Method	Purpose	Returns
`sendKeys("text")`	Type text into a field	nothing
`click()`	Click a button, link, checkbox, radio	nothing
`getText()`	Visible text of the element	`String`
`getAttribute("placeholder")`	Value of an attribute	`String`

Method	Purpose	Returns
`isDisplayed()`	Is it visible on screen?	`boolean`
`isEnabled()`	Can the user interact with it?	`boolean`

Common extras from standard Selenium (the Week 4 outline implies them, but your files do not show them): `clear()` empties a text field, `isSelected()` checks a checkbox or radio button, and the `Select` class handles `<select>` dropdowns.

```
// Type into a search box and click search (Week 2.2 handout)
WebElement box = driver.findElement(By.name("home-query"));
box.sendKeys("toyota");                                    // type
String hint = box.getAttribute("placeholder");             // read an attribute
driver.findElement(By.id("home-search-btn")).click();      // click

// Standard dropdown (outline only)
Select country = new Select(driver.findElement(By.id("country")));
country.selectByVisibleText("Pakistan");
```

Highlight helper from your handout (draws a red border so you can see the element, then pauses):

```
public void highlightElement(WebElement element) throws InterruptedException
{
    JavascriptExecutor js = (JavascriptExecutor) driver;     // lets Selenium
run JavaScript
    js.executeScript("arguments[0].style.border='6px solid red'", element);
    Thread.sleep(3000);                                      // demo only,
see 1.12
}
```

File upload (the correct way). A file input is `<input type="file">`. You do **not** click it and open the operating system dialog. You send the **file path** as text:

```
WebElement file = driver.findElement(By.id("uploadFile"));
file.sendKeys("C:\\Users\\you\\Desktop\\test.txt");   // sets the chosen file
driver.findElement(By.id("submitButton")).click();     // then click the real
upload button
```

`click()` vs `submit()`. `submit()` is meant for a `<form>` element, or an element **inside** a form. It submits that form. It is an older method; clicking the real submit button is the normal approach because it behaves like a real user.

Common mistakes.

- Locating an element and forgetting to act on it (no `click()` or `sendKeys()`).
- Calling `getText()` on an `<input>` and getting an empty string. For input values use `getAttribute("value")`.
- Calling `submit()` on a file input (see Part 2, issue 3).
- Sending a relative or wrong file path to a file input. Use a full path that exists on the machine.
- Acting on an element before the page has finished loading (see Waits, 1.12).
- Using `isDisplayed()` on an element that does not exist in the DOM. That throws `NoSuchElementException`; it does not return `false`.

1.10 TestNG Annotations (Week 2; Week 6 outline)

Concept. An *annotation* is a metadata tag (starting with `@`) that tells TestNG how to treat a method. TestNG decides **when** each method runs; you do not call them yourself.

Why it is needed. Without annotations you would have a `main` method and call everything by hand. With them you get set-up and clean-up that run at the right time, and you can run, group and report many tests.

Key syntax: annotations in your slides.

Annotation	When it runs
`@Test`	Marks a method as a test case
`@BeforeClass`	**Once** before the first `@Test` in the class
`@AfterClass`	**Once** after the last `@Test` in the class
`@BeforeMethod`	Before **each** `@Test` method
`@AfterMethod`	After **each** `@Test` method

Other common ones (standard TestNG, not on your slides): `@BeforeTest`, `@AfterTest`, `@BeforeSuite`, `@AfterSuite`. The Week 6 outline also names **TestNG XML suites**, which list classes to run together.

Execution flow with two tests, `loginTest` and `logoutTest`:

```
@BeforeClass   -> runs once
  @BeforeMethod -> @Test loginTest  -> @AfterMethod
  @BeforeMethod -> @Test logoutTest -> @AfterMethod
@AfterClass    -> runs once
```

TestNG runs test methods in **alphabetical order** by default, not the order you wrote them. If you need a fixed order, use `@Test(priority = 1)`, `@Test(priority = 2)` and so on.

```
@BeforeMethod
public void setUp()    { System.out.println("Setup"); }     // fresh start
for every test
@Test
public void loginTest() { System.out.println("Test"); }
@AfterMethod
public void tearDown() { System.out.println("Cleanup"); }
```

Common mistakes.

- Confusing `@BeforeClass` (once) with `@BeforeMethod` (before every test).
- Creating the `WebDriver` inside a `@Test` method, so every test opens its own browser and nothing is shared.
- Assuming tests run in the order they appear in the file.
- Forgetting `import org.testng.annotations.*;`, or importing JUnit's `@Test` by mistake.
- Using `@AfterClass` clean-up that calls `driver.quit()` when the driver was never created. That causes a `NullPointerException`.

1.11 Assertions (Week 6 outline only)

Concept. An *assertion* is a check inside a test. If the check is false, TestNG stops that test and marks it **failed**. If you have no assertion, a test only fails when the code throws an exception. So a test with no assertion may "pass" even when the page is wrong.

Why it is needed. Automation must decide pass or fail by itself. Assertions compare **actual** and **expected** results. Printing the title with `System.out.println` is not a check.

Key syntax (`org.testng.Assert`; the argument order is **actual, expected, message**):

```
import org.testng.Assert;

Assert.assertEquals(driver.getTitle(), "Automation Exercise", "Wrong title");
Assert.assertTrue(driver.findElement(By.id("sale_image")).isDisplayed());
Assert.assertNotEquals(actual, unexpected);
Assert.assertNull(object);
```

Type	Behaviour on failure
Hard assert (`Assert`)	Stops the test at once; later lines do not run

Type	Behaviour on failure
Soft assert (`SoftAssert`)	Records the failure, continues, reports all at `assertAll()`

Common mistakes.

- Swapping actual and expected, which makes the failure message misleading.
- Writing a test with no assertion at all.
- Using `SoftAssert` and forgetting `assertAll()` . Then the test passes even when checks failed.
- Asserting on `driver.getTitle()` before the page has loaded.
- For the exam: Week 6 also lists TestNG XML suites and report review. Practise creating a `testng.xml` that lists your classes and running it from IntelliJ.

1.12 Waits and Synchronization (Week 5 outline; Thread.sleep appears in Week 2)

Concept. *Synchronization* means making the test wait until the page is ready. Web pages load at uneven speeds. If Selenium looks for an element before it exists, the test fails even though the app is fine. That is the "false positive" in your Week 1 challenges slide.

Why it is needed. Stable tests do not depend on how fast the network is on a given day.

Key syntax: the three kinds of wait.

Wait	How it works	Good for	Weak point
`Thread.sleep(ms)`	Pauses Java for a **fixed** time	Quick demos only	Always waits the full time; too short still fails
Implicit wait	Set once; every `findElement` keeps retrying up to the limit	Simple, global	Applies to every lookup; hard to tune
Explicit wait	Waits for **one specific condition**, then continues at once	Real tests	Needs a little more code

```
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;

// Implicit wait: set once after creating the driver
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

// Explicit wait: wait for this element only, up to 10 seconds
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement btn =
wait.until(ExpectedConditions.elementToBeClickable(By.id("login")));
btn.click();
```

A third type, *fluent wait*, is an explicit wait where you also set how often to re-check and which errors to ignore. It is standard Selenium; your outline names only implicit and explicit waits.

Common mistakes.

- Filling a test with `Thread.sleep(3000)` (the Week 2 handouts do this for demo purposes, not as good practice).
- Mixing implicit and explicit waits. Their times can add up in confusing ways, so pick explicit waits for real tests.
- Passing a number instead of `Duration.ofSeconds(...)`. Selenium 4 expects a `Duration`.
- Forgetting that `Thread.sleep` throws `InterruptedException`, so the method needs `throws InterruptedException` or a try-catch.

1.13 Actions Class (Week 5 outline only)

Concept. The `Actions` class simulates complex mouse and keyboard actions that `click()` and `sendKeys()` cannot do alone: hover, double-click, right-click, drag and drop, scrolling.

Key syntax. Build the chain, then call `perform()`; without `perform()` nothing happens.

```
Actions act = new Actions(driver);
WebElement menu = driver.findElement(By.id("menu"));

act.moveToElement(menu).perform();          // hover
act.doubleClick(menu).perform();            // double-click
act.contextClick(menu).perform();           // right-click
act.moveToElement(menu).click().perform();  // hover then click (a chain)
```

Common mistakes. Forgetting `.perform()`. Using `click()` when a hover menu must open first. Not importing `org.openqa.selenium.interactions.Actions`.

1.14 Page Object Model (Week 7 outline only)

Concept. *Page Object Model (POM)* is a design pattern. Each web page gets its own Java class (a *page class*) that holds that page's locators and the actions you can do on it. The test class holds only the test steps and checks.

Why it is needed. If a locator changes, you fix it in **one** page class instead of in every test. Tests become short, readable and reusable. This answers the *maintenance overhead* challenge from Week 1.

Key syntax. The outline names `PageFactory`, which creates the elements from `@FindBy` annotations, and a `configReader` for settings such as the URL.

```
public class LoginPage {
    WebDriver driver;

    @FindBy(id = "email")    WebElement emailField;    // locator lives here
only
    @FindBy(id = "password") WebElement passwordField;
    @FindBy(id = "login")    WebElement loginButton;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);        // connects the fields
to the page
    }

    public void login(String user, String pass) {      // one reusable action
        emailField.sendKeys(user);
        passwordField.sendKeys(pass);
        loginButton.click();
    }
}

// In the test class:
LoginPage login = new LoginPage(driver);
login.login("sample@user.com", "123456789");
Assert.assertTrue(driver.getCurrentUrl().contains("automationexercise"));
```

Common mistakes.

- Putting assertions and locators in the same class and calling it "POM".

- Forgetting `PageFactory.initElements(...)`. Fields stay `null` and you get a `NullPointerException`.
- Creating a new `WebDriver` inside each page class. Pass **one** driver into the constructor.
- Hardcoding URLs and passwords in the test instead of reading them from a config file.

PART 2: LINE-BY-LINE EXPLANATION OF YOUR CODE

Important. Your two Java files were not attached, so the code below is a **reconstruction** from your description: the same methods, the same four bugs you listed, in the most likely order. Website URLs, the file path and some locator values are examples I chose. If your real file differs, paste it and I will redo this table against your exact lines.

2.1 Class `week_3_ATS` (navigation and windows)

```
01 package tests;
02
03 import java.sql.Driver;                          // unused import
04 import org.openqa.selenium.WebDriver;
05 import org.openqa.selenium.WindowType;
06 import org.openqa.selenium.chrome.ChromeDriver;
07 import org.testng.annotations.Test;
08
09 public class week_3_ATS {
10     WebDriver driver = new ChromeDriver();     // field-level driver
11
12     @Test
13     public void navigationTest() throws InterruptedException {
14         driver.get("https://www.automationexercise.com");
15         String parent = driver.getWindowHandle();
16         System.out.println("Parent handle: " + parent);
17         driver.navigate().to("");                  // empty URL
18         Thread.sleep(3000);
19         driver.navigate().back();
20         Thread.sleep(3000);
21         driver.navigate().refresh();
22         Thread.sleep(3000);
23         driver.navigate().forward();
24         Thread.sleep(3000);
25         driver.switchTo().newWindow(WindowType.TAB);
26         driver.get("https://www.daraz.pk");
27         Thread.sleep(3000);
28         driver.switchTo().window(parent);
```

```
29          Thread.sleep(3000);
30          driver.close();
31      }
32  }
```

Line	What it does	Expected behaviour or problem
1	Puts the class in package `tests`	Must match the folder under `src/test/java`
3	Imports `java.sql.Driver`	**Bug:** a database class that is never used. It should be `WebDriver` only, and that is already imported on line 4
4	Imports the `WebDriver` interface	Needed for the field on line 10
5	Imports `WindowType`	Needed for `WindowType.TAB` on line 25
6	Imports `ChromeDriver`	Lets you create a Chrome browser
7	Imports TestNG `@Test`	TestNG will run annotated methods
9	Declares the class	**Bad practice:** the name `week_3_ATS` does not follow Java style. Class names should be `Week3AtsTest` (capital letters, no underscores)
10	Creates a `ChromeDriver` as a **field**	**Bug:** the browser opens as soon as the object is created, before any test runs, and is never closed properly. See issue 5
12	`@Test`	Marks line 13's method as a test
13	Test method; `throws InterruptedException`	Required only because of `Thread.sleep`
14	`driver.get(url)`	Opens automationexercise.com and waits for the page to load
15	`getWindowHandle()`	Saves the current tab's ID in `parent`. Good: you need it to come back later

Line	What it does	Expected behaviour or problem
16	Prints the handle	Output looks like `Parent handle: 8F3A...`. It is different every run
17	`navigate().to("")`	**Bug:** empty URL. The test may throw an exception here and stop; if it does not, the history is not what lines 19 and 23 expect. See issue 1
18, 20, 22, 24, 27, 29	`Thread.sleep(3000)`	**Bad practice:** fixed 3-second pauses. Six of them add 18 seconds and still do not guarantee the page is ready. See issue 4
19	`navigate().back()`	Goes one page back in history. Only useful if a second page was visited first
21	`navigate().refresh()`	Reloads the current page
23	`navigate().forward()`	Goes one page forward. Does nothing if there is no forward page
25	`switchTo().newWindow(WindowType.TAB)`	Opens a new blank tab and **moves focus to it**
26	`get(daraz)`	Loads Daraz in the **new** tab, because focus is there
28	`switchTo().window(parent)`	Moves focus back to the first tab using the saved handle
30	`driver.close()`	Closes the **current** tab (the first one). The Daraz tab stays open and the driver session is not ended. See issue 6

There is also **no assertion** anywhere. The test can pass without proving that any page was correct (issue 7).

2.2 Class `Locators` (By.name, By.id, sendKeys, upload)

```
01 package tests;
02
03 import org.openqa.selenium.By;
04 import org.openqa.selenium.WebDriver;
05 import org.openqa.selenium.WebElement;
06 import org.openqa.selenium.chrome.ChromeDriver;
07 import org.testng.annotations.Test;
08
09 public class Locators {
10     WebDriver driver = new ChromeDriver();     // a second field-level
driver
11
12     @Test
13     public void locatorTest() throws InterruptedException {
14         driver.get("https://www.automationexercise.com/contact_us");
15         driver.findElement(By.name("name")).sendKeys("Ali Khan");
16         driver.findElement(By.name("email")).sendKeys("ali@example.com");
17
driver.findElement(By.id("susbscribe_email")).sendKeys("ali@example.com");
18         WebElement file = driver.findElement(By.name("upload_file"));
19         file.sendKeys("C:\\Users\\Ali\\Desktop\\test.txt");
20         file.submit();                          // submit on a file input
21         Thread.sleep(3000);
22     }
23 }
```

Line	What it does	Expected behaviour or problem
3 to 7	Imports `By`, `WebDriver`, `WebElement`, `ChromeDriver`, `@Test`	All are used. This file is clean
9	Class `Locators`	Name is fine (capital first letter)
10	Field-level `new ChromeDriver()`	**Bug:** same as in `week_3_ATS`. Run both classes together and you get **two browsers**, each created at field level
14	Opens the Contact Us page	The page loads
15	`findElement(By.name("name"))` then `sendKeys`	Finds the first element with `name="name"` and types the text. Matches the handout's Name input
16	Same with `By.name("email")`	Types the email into the email input

Line	What it does	Expected behaviour or problem
17	`By.id("susbscribe_email")`	Finds the footer subscribe box by its id and types. (The spelling `susbscribe` is how the handout writes the id; copy it exactly from the page)
18	Stores the file input in a `WebElement` variable	Good: you reuse it on lines 19 and 20 instead of searching twice
19	`sendKeys(path)` on the file input	**Correct way to upload:** it sets the chosen file. The path must exist on the machine running the test
20	`file.submit()`	**Bug:** you submit through the file input instead of clicking the real submit button. See issue 3
21	`Thread.sleep(3000)`	Pause so you can see the result. Same weakness as above
none	No `quit()` and no assertion	The browser stays open and nothing is checked

2.3 Bugs and Bad Practices, with Fixes

#	Issue	Severity
1	Empty `driver.navigate().to("")`	Bug, can fail the test
2	Unused `import java.sql.Driver`	Clean-up, wrong class if used by mistake
3	`file.submit()` on a file input	Bug, fragile and not user-like
4	Hardcoded `Thread.sleep()`	Bad practice, slow and unreliable
5	`WebDriver` created at field level in several classes	Bad practice, leaks browsers
6	`close()` used instead of `quit()`, no clean-up method	Bug, leaves browser and driver running
7	No assertions	Test cannot detect a wrong page
8	Class name `week_3_ATS`	Style

Issue 1: empty `navigate().to("")`

- **Problem:** the URL argument is an empty string.
- **Why it is wrong:** there is no page to open. Depending on browser and driver version, this throws an exception (the test stops there) or does nothing. Either way the later `back()` and `forward()` calls do not test what you meant, because the browser history is not what you think.
- **Fix:** pass a full, valid URL, or delete the line.

```
// Before
driver.navigate().to("");
// After
driver.navigate().to("https://www.daraz.pk");   // a real second page, so
back() has somewhere to go
```

Issue 2: unused `import java.sql.Driver`

- **Problem:** `java.sql.Driver` is a JDBC (database) interface. Your test never uses it.
- **Why it is wrong:** it adds noise and is a classic IDE auto-import mistake, since it is easy to pick by mistake when you type `Driver`. If you ever use the name `Driver` later, it points to the wrong type and the code will not compile or behave as expected. Reviewers read it as sloppy code.
- **Fix:** delete it. You only need `org.openqa.selenium.WebDriver`. In IntelliJ use Code > Optimize Imports (Ctrl+Alt+O).

```
// Before
import java.sql.Driver;
// After: line removed
```

Issue 3: `file.submit()` on a file input

- **Problem:** the test calls `submit()` on the `<input type="file">` element.
- **Why it is wrong:** `submit()` only works on a form, or on an element **inside** a form; it then submits that form. If the file input is not inside a `<form>`, Selenium throws `NoSuchElementException` (no parent form). Even when it works, it **skips clicking the real Submit button**, so the page's button logic and validation are not tested the way a user triggers them. Choosing a file is done with `sendKeys(path)`, and sending the form is done by clicking its button.
- **Fix:**

```
// Before
file.sendKeys("C:\\Users\\Ali\\Desktop\\test.txt");
file.submit();
// After
file.sendKeys(path);                                    // choose the file
driver.findElement(By.name("submit")).click();          // click the real
button
```

Issue 4: hardcoded `Thread.sleep()`

- **Problem:** fixed 3-second pauses after almost every line.
- **Why it is wrong:** too short on a slow network (test fails), too long on a fast one (time is wasted). Six pauses add 18 seconds to one test. It also forces `throws InterruptedException` on every method.
- **Fix:** wait for a condition instead (explicit wait, section 1.12).

```
// Before
driver.navigate().back();
Thread.sleep(3000);
// After
driver.navigate().back();
new WebDriverWait(driver, Duration.ofSeconds(10))
        .until(ExpectedConditions.urlContains("automationexercise"));
```

For the exam: the course handouts also use `Thread.sleep` to slow things down so you can watch. That is acceptable for a demo, not for a real test.

Issue 5: `WebDriver` created at field level in several classes

- **Problem:** `WebDriver driver = new ChromeDriver();` sits in both `week_3_ATS` and `Locators`.
- **Why it is wrong:** the browser opens when the class is created, not when a test needs it, so running the suite opens extra browsers. Nothing closes them. The set-up is copied into every class, so a change (for example Firefox) must be made in many places. If creation fails, the error appears at class construction, which is confusing. Your lecture sets the driver up inside `@BeforeClass` with `quit()` in `@AfterClass`; this code skips that pattern.
- **Fix:** declare the field without creating it, and create and close it in one shared base class.

Issue 6: `close()` with no clean-up method

- **Problem:** the test ends with `driver.close()`; nothing calls `quit()`.

- **Why it is wrong:** `close()` shuts only the current tab. The Daraz tab and the driver process stay alive.
- **Fix:** put `driver.quit()` in `@AfterMethod` (or `@AfterClass`), as shown below.

Issue 7: no assertions

- **Problem:** the test prints and navigates but never checks anything.
- **Why it is wrong:** it passes as long as no exception is thrown, even if you landed on the wrong page.
- **Fix:** add `Assert.assertTrue(...)` or `Assert.assertEquals(...)` after the key steps.

Issue 8: class name

- **Problem:** `week_3_ATS` uses lowercase and underscores.
- **Fix:** `Week3AtsTest`. Java classes use *PascalCase*.

2.4 Corrected Version of Both Classes

One base class owns the browser. Each test class only contains test steps.

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;

public class BaseTest {
    protected WebDriver driver;                 // declared only, not created
here

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();            // one fresh browser per test
        driver.manage().window().maximize();
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver != null) driver.quit();      // closes every tab, ends the
session
    }
}
```

```
package tests;

import org.openqa.selenium.WindowType;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.Test;
import java.time.Duration;

public class Week3AtsTest extends BaseTest {

    @Test
    public void navigationAndTabsTest() {
        WebDriverWait wait = new WebDriverWait(driver,
Duration.ofSeconds(10));

        driver.get("https://www.automationexercise.com");
        String parent = driver.getWindowHandle();               // save first
tab

        driver.navigate().to("https://www.daraz.pk");           // valid
second page
        driver.navigate().back();
        wait.until(ExpectedConditions.urlContains("automationexercise"));
        driver.navigate().forward();
        driver.navigate().refresh();

        driver.switchTo().newWindow(WindowType.TAB);            // focus
moves to new tab
        driver.get("https://www.automationexercise.com/products");
        Assert.assertEquals(driver.getWindowHandles().size(), 2, "Two tabs
expected");

        driver.close();                                         // closes the
new tab only
        driver.switchTo().window(parent);                       // must
switch back
        Assert.assertTrue(driver.getCurrentUrl().contains("daraz"));
    }
}
```

```
package tests;

import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.testng.annotations.Test;
```

```
public class LocatorsTest extends BaseTest {

    @Test
    public void contactFormTest() {
        driver.get("https://www.automationexercise.com/contact_us");

        driver.findElement(By.name("name")).sendKeys("Ali Khan");
        driver.findElement(By.name("email")).sendKeys("ali@example.com");

        String path = System.getProperty("user.dir") +
"/src/test/resources/test.txt";
        WebElement file = driver.findElement(By.name("upload_file"));
        file.sendKeys(path);                                    // choose the
file

        driver.findElement(By.name("submit")).click();          // click the
real button
    }
}
```

Check the locator values and the file path against the real page and your own folder; I chose them as examples.

PART 3: EXAM-ORIENTED SUMMARY

3.1 Quick-Revision Cheat Sheet

WebDriver commands

Task	Command	Returns
Open browser	`WebDriver driver = new ChromeDriver();`	driver
Open URL	`driver.get(url)` or `driver.navigate().to(url)`	void
Back, forward, refresh	`navigate().back()`, `.forward()`, `.refresh()`	void
Page title, URL	`getTitle()`, `getCurrentUrl()`	`String`
Current tab ID	`getWindowHandle()`	`String`
All tab IDs	`getWindowHandles()`	`Set<String>`

Task	Command	Returns
New tab	`switchTo().newWindow(WindowType.TAB)`	focus moves to it
Switch tab	`switchTo().window(handle)`	focus moves
Close current tab	`close()`	void
End everything	`quit()`	void
Find one element	`findElement(By...)`	`WebElement` (or exception)
Find many	`findElements(By...)`	`List<WebElement>` (maybe empty)
Click, type	`click()`, `sendKeys(text)`	void
Read	`getText()`, `getAttribute("x")`	`String`
State	`isDisplayed()`, `isEnabled()`	`boolean`
Window size	`manage().window().maximize()` / `.minimize()`	void

Common exceptions and their usual cause

Exception	Usual cause
`NoSuchElementException`	Wrong locator, or element not loaded yet
`NoSuchWindowException`	Used the driver after `close()` without switching windows
`NullPointerException` on `driver`	Declared the driver but never wrote `new ChromeDriver()`, or `quit()` ran on a null driver
`TimeoutException`	Explicit wait condition not met in time
`InterruptedException`	`Thread.sleep()` needs `throws` or try-catch

TestNG order and Maven

Item	Rule
Order of annotations	`@BeforeClass` (once) > `@BeforeMethod` > `@Test` > `@AfterMethod` > ... > `@AfterClass` (once)

Item	Rule
Default test order	Alphabetical by method name; use `priority` to control it
`pom.xml`	Lists dependencies; reload Maven after editing
Scope `compile` / `test`	Everywhere / only in `src/test/java`
Folders	`src/main/java` app code, `src/test/java` test code

Locator priority (Week 3)

Rank	Locator	Use when
1	ID	Unique id exists
2	Name	Unique name exists (forms)
3	Class name	One simple unique class
4	Link text, partial link text	`<a>` links only
5	CSS	Flexible, attribute-based
6	XPath	Nothing else works; text or parent-child needed

Key definitions

Term	One-line meaning
Regression testing	Re-testing to confirm old features still work after a change
DOM	Browser's tree of objects built from the HTML
Locator	Instruction that tells Selenium which element to use
Window handle	Unique text ID of a browser window or tab
Assertion	A check that fails the test if false
Implicit wait	One global wait applied to every `findElement`
Explicit wait	Wait for one named condition
POM	One class per page holding its locators and actions

Term	One-line meaning
Annotation	`@` tag that tells TestNG how to run a method

Likely imports

```
import org.openqa.selenium.*;                          // By, WebDriver,
WebElement, WindowType
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;               // WebDriverWait,
ExpectedConditions, Select
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
```

3.2 Ten Likely MCQs

Q1. Which statement about automation testing versus manual testing is correct?

- A) Automation has no setup cost
- B) Automation has a higher initial cost but is faster for repeated tests
- C) Manual testing is faster for 100 repeated checks
- D) Automation can judge how user-friendly a design feels

Answer: B. Scripts take time to build, then repeat quickly. UI/UX feel still needs a human.

Q2. What is the main purpose of `pom.xml` in a Maven project?

- A) To store the test results
- B) To hold the Java test classes
- C) To list dependencies such as Selenium and TestNG
- D) To launch the browser

Answer: C. Maven reads `pom.xml` and downloads the libraries you list there.

Q3. Which locator is generally the fastest and most reliable?

- A) ID
- B) XPath
- C) Partial link text
- D) Tag name

Answer: A. IDs are supposed to be unique on a page, so the match is direct.

Q4. In real Selenium, what does `findElement()` do when no element matches?

- A) Returns `null`
- B) Returns an empty list
- C) Returns `false`
- D) Throws `NoSuchElementException`

Answer: D. `findElements()` returns an empty list instead. Your Week 3 slide 10 says `null`; if your quiz key follows the slide, ask your teacher which form she wants.

Q5. When does a method marked `@BeforeClass` run?

- A) Before every `@Test` method
- B) Once, before the first `@Test` method in the class
- C) After every `@Test` method
- D) Once, after the last `@Test` method

Answer: B. `@BeforeMethod` is the one that runs before each test.

Q6. Which line opens a new browser tab and moves Selenium's focus to it?

- A) `driver.get("about:blank")`
- B) `driver.switchTo().window(handle)`
- C) `driver.switchTo().newWindow(WindowType.TAB)`
- D) `driver.navigate().to("newtab")`

Answer: C. It both creates the tab and switches to it.

Q7. What is the difference between `close()` and `quit()`?

- A) `close()` closes the current window; `quit()` closes all windows and ends the session
- B) `close()` ends the session; `quit()` closes one window
- C) They are identical
- D) `quit()` only works in Firefox

Answer: A. Use `quit()` for final clean-up.

Q8. What does the CSS selector `input[placeholder^='Sub']` find?

- A) Inputs whose placeholder ends with Sub
- B) Inputs whose placeholder contains Sub anywhere
- C) Inputs whose id is Sub
- D) Inputs whose placeholder starts with Sub

Answer: D. `^=` means starts with, `$=` ends with, `*=` contains.

Q9. Why is `Thread.sleep(3000)` a poor way to wait for a page?

- A) It is not valid Java
- B) It always waits the full time and does not check whether the page is ready
- C) It only works with Firefox
- D) It closes the browser

Answer: B. An explicit wait continues as soon as its condition is met.

Q10. What is the correct way to choose a file in `<input type="file">`?

- A) `file.click()` and use the system dialog
- B) `file.submit()`
- C) `file.sendKeys("full file path")`
- D) `driver.get("full file path")`

Answer: C. Send the path as text, then click the page's real upload or submit button.

3.3 Five Short-Answer Questions with Model Answers

SQ1. Compare manual and automated testing.

In manual testing a tester runs the test cases by hand, while in automated testing scripts run them and compare actual results with expected results. Automation is faster and more consistent for repeated and regression tests, and scripts can be reused for every release. Manual testing has a lower starting cost and is better for new features and for judging how a design feels. Automation needs a higher initial investment in tools and skills and its scripts must be updated when the page changes. Teams usually use both: automation for stable repeated checks, manual testing for exploration.

SQ2. What is the difference between `driver.close()` and `driver.quit()`? Which one belongs in clean-up code?

`close()` closes only the window or tab that Selenium is currently controlling. `quit()` closes every window opened by the session and ends the WebDriver session, so the driver process stops too. If you call `close()` on the only window, the browser closes but the session can remain; if other tabs are open they stay open. For this reason clean-up code (`@AfterMethod` or `@AfterClass`) should call `quit()`. After `close()`, you must call `switchTo().window(handle)` before using the driver again.

SQ3. What is a locator? Name four locator types and say which you would try first.

A locator tells Selenium which exact element on the page to work with, because Selenium reads the DOM and needs a precise description. Locator types include ID, Name, Class Name, Tag Name, Link Text, CSS selector and XPath. I would try ID first because it is meant

to be unique, fast and reliable. If there is no unique id, I would try Name, then Class Name, then a CSS selector, and I would use XPath last, for cases such as matching text or moving between parent and child elements. XPath is powerful but slower and more brittle.

SQ4. How does Selenium handle more than one browser tab?

Every window or tab has a unique text ID called a window handle, and Selenium controls only one window at a time. `getWindowHandle()` returns the current handle and `getWindowHandles()` returns all of them as a `Set<String>`. To work in another tab you call `driver.switchTo().window(handle)`, or you call `switchTo().newWindow(WindowType.TAB)`, which opens a new tab and moves focus to it automatically. You should save the original handle first so you can return to it, especially after `close()`.

SQ5. Explain the TestNG annotations `@BeforeClass`, `@BeforeMethod`, `@Test`, `@AfterMethod` and `@AfterClass`.

`@Test` marks a method as a test case. `@BeforeClass` and `@AfterClass` run once, before the first test and after the last test of the class, so they suit a one-time browser start and quit. `@BeforeMethod` and `@AfterMethod` run before and after every single test method, so they suit a fresh start for each test. TestNG controls this order, so you never call these methods yourself. Creating the `WebDriver` in a set-up method and calling `quit()` in a tear-down method keeps the tests short and avoids leaving browsers open.

PART 4: RECOMMENDED PRACTICE TASKS

Your midterm is practical, so type these yourself in IntelliJ instead of copying. Use automationexercise.com because your handouts use it. Each task builds on the `BaseTest` class from section 2.4.

Task 1: Navigation and history (concept: browser navigation)

- **Goal:** open three different pages, then move through history with `back()`, `forward()` and `refresh()`.
- **Steps:** `get()` the home page, `navigate().to()` the products page, `navigate().to()` the contact page. Call `back()` twice and print `getCurrentUrl()` after each call. Then `forward()` once and `refresh()`.
- **Expected outcome:** the printed URLs are products, then home (after the two back calls), then products again (after forward). The test has an assertion for each URL and no `Thread.sleep`.

Task 2: Tabs and window handles (concept: window handling)

- **Goal:** work in two tabs and return to the first.

- **Steps:** save `getWindowHandle()`. Open a new tab with `switchTo().newWindow(WindowType.TAB)`, load another site, and print `getTitle()`. Assert that `getWindowHandles().size()` is 2. Call `close()`, switch back with `switchTo().window(original)`, and print the first tab's title.
- **Expected outcome:** two titles print, the handle count is 2 before closing and 1 after, and no `NoSuchWindowException` appears. Then try removing the `switchTo().window` line to see that error yourself.

Task 3: Locator table (concept: locators)

- **Goal:** find the same element in several ways.
- **Steps:** on the Contact Us page, find the Name input with `By.name`, `By.cssSelector("input[name='name']")` and `By.xpath("//input[@name='name']")`. Also use `findElements(By.tagName("input"))` and print how many there are. Fill in the "From students" rows of your Week 3 handout with your own locators.
- **Expected outcome:** all three locators return the same element (compare with `equals`), the count prints, and you can say which one you prefer and why. Test each locator in Developer Tools before coding it.

Task 4: Fill a form and upload a file (concept: element interaction)

- **Goal:** complete a form the way a user would, including the file upload that was wrong in Part 2.
- **Steps:** fill name, email, subject and message with `sendKeys`. Create `src/test/resources/test.txt`. Send its full path to the file input with `sendKeys`. Click the Submit button with `click()`. Read `getAttribute("value")` from one field and assert it matches what you typed.
- **Expected outcome:** the form is submitted through the button, not through `submit()`, and the assertion passes. Handle any pop-up that the page shows before checking the result.

Task 5: Lifecycle, waits and assertions (concepts: TestNG annotations, waits, assertions)

- **Goal:** turn a messy test into a clean one.
- **Steps:** take your code from Task 1 and add `@BeforeMethod` and `@AfterMethod` print statements to see the order. Add a second `@Test` method and watch the order TestNG picks, then fix it with `priority`. Replace every `Thread.sleep` with `WebDriverWait` and `ExpectedConditions`. Add one failing assertion on purpose and read the failure message.

- **Expected outcome:** the console shows set-up and tear-down around **each** test, there is no `Thread.sleep` in the file, the run time drops, and the deliberate failure shows your own message text.